

TRABAJO FINAL INTEGRADOR PROGRAMACIÓN III

FOODSTORE - Sistema de Gestion de Pedidos

Profesor: Ariel Enferrel

Tutor: Juan Cruz Robledo

Alumno:

Cuquejo, Mauro Maximiliano

Legajo: 56370

Comisión:12

Fecha de Entrega: 02/07/2026

ÍNDICE DE CONTENIDOS

Introducción	3
Marco Teórico	4
Decisiones Técnicas y Arquitectura	5
Dificultades y Soluciones.....	10
Conclusiones.....	12
Bibliografía	13
Link a Youtube	13
Link a Repositorios	13

Introducción

Para el trabajo práctico integrador final de la materia programación 3, se desarrolló el sistema de gestión de pedidos **FoodStore**. El mismo permite la carga de usuarios administradores y clientes. Los primeros, serán los encargados de administrar las existencias de productos, así como organizarlos por categorías y gestionar el estado de los pedidos de los clientes. A nivel de cliente, es posible navegar por la página de la tienda, en donde podrá visualizar la totalidad de los productos en existencia, filtrarlos u ordenarlos por distintas propiedades y proceder a la compra de uno o varios de ellos. Una vez realizado el pedido, podrá consultar el estado del mismo desde la navegación correspondiente.

El proyecto consta de un **Backend** y un **Frontend** que, aunque en esta etapa se encuentran desconectados, están preparados para conectarse en una etapa posterior. El Backend está desarrollado en Java 21, utilizando JPA e Hibernate para la capa de persistencia de datos. El mismo se administra actualmente por consola y gestiona una base de datos H2. El Frontend, está desarrollado con TypeScript vanilla y Vite. Para el manejo de datos, se utiliza el LocalStorage y la carga inicial es provista por los profesores, a través de JSON, a los que se accede mediante fetch en un servicio preparado para ser conectado posteriormente al Backend.

Marco Teórico

Tecnologías del Backend

- **Java 21:** Lenguaje principal de desarrollo backend. Se seleccionó por su fuerte tipado estático, robustez, y soporte para programación orientada a objetos moderna y expresiones lambda usadas en flujos de datos.
- **JPA - Hibernate:** Estándar de persistencia en Java que actúa como ORM. Permite interactuar con la base de datos relacional modelando las tablas como entidades/clases de Java, abstrayendo las consultas SQL directas mediante JPQL.
- **H2 Database (Modo Archivo):** Base de datos relacional SQL en memoria/archivo ligera y rápida, ideal para entornos de desarrollo y pruebas locales. Persiste los datos en un archivo local `./data/jpa_db` sin requerir la instalación de un servidor de base de datos externo.
- **Gradle:** Herramienta de automatización de compilación y gestor de dependencias que facilita la configuración de dependencias de Jakarta Persistence, Hibernate y H2.

Tecnologías del Frontend

- **Vite:** Herramienta de compilación ultrarrápida para aplicaciones web modernas.
- **TypeScript:** Superset de JavaScript que añade tipado estático y reduce significativamente los errores en tiempo de ejecución.
- **HTML5 y CSS3 (Vanilla CSS):** Utilizados para configurar y maquetar visualmente la interfaz de la tienda virtual, logrando un diseño fluido, moderno y responsivo adaptado a múltiples dispositivos.

Conceptos Clave Aplicados

- **ORM (Object-Relational Mapping):** Nos permite asociar las tablas de la base de datos relacional a objetos de código de software.
- **Persistencia en Cascada:** Configuración de JPA que propaga las operaciones realizadas en una entidad principal (como Pedido) a sus entidades hijas (como DetallePedido) automáticamente.
- **Baja Lógica:** En lugar de eliminar registros físicamente con el comando SQL DELETE (lo cual rompería el historial histórico de ventas), se aplica una baja lógica asignando un flag *eliminado* = *true* para ocultarlos de los listados activos de cara al usuario.

Decisiones Técnicas y Arquitectura

Arquitectura General y Capas

El sistema Backend se estructuró siguiendo un patrón de arquitectura limpia y desacoplada, separando la interfaz de usuario (Consola) de la lógica de negocio y persistencia:

Backend

- **Capa de Presentación / Control (Main.java):** Se encarga exclusivamente de interactuar con el usuario mediante la entrada y salida de datos por consola. No maneja lógica transaccional ni accede directamente a JPA, delegando estas acciones a los repositorios correspondientes.
- **Capa de Persistencia (Repositories):** Encapsula el acceso a la base de datos relacional. Cada repositorio abre y cierra su propio EntityManager localmente en bloques finally.
- **Capa del Modelo (Entities / Enums):** Define la estructura de datos que representa nuestro dominio de negocio (Usuario, Pedido, Producto, Categoria, DetallePedido) y sus estados permitidos.

Frontend:

- **Aplicación Basada en Vanilla TS + Vite:** Este proyecto funciona como una aplicación multi-página clásica pero potenciada por Vite y TypeScript.
- **Navegación y Enrutamiento Físico:** La navegación se realiza mediante la redirección directa del navegador a través de window.location.href, implementada en navigate.ts. Las rutas válidas y los accesos permitidos se centralizan como constantes en routes.ts.
- **Persistencia de Datos en LocalStorage:** Al iniciar la aplicación, los datos por defecto se obtienen mediante peticiones fetch de archivos JSON estáticos en public/data/ (por ejemplo, categorías, usuarios y productos) definidos en fetch.ts. Tras la primera carga, toda la manipulación de base de datos (lectura, escritura, actualizaciones y filtrados de negocio) se realiza y persiste en la sesión del cliente usando localStorage a través de wrappers genéricos en localStorage.ts.
- **Middleware de Seguridad / Guard de Rutas:** En auth.ts se implementa un guard global (guardRoutes) que valida en cada carga si hay un usuario logueado en la sesión y comprueba su rol (ADMIN o USUARIO). Redirige automáticamente al login o a la página correspondiente en caso de intento de acceso no autorizado.

- **Estructura de Directorios Orientada a Características**

- **src/pages/**: Agrupadas por contexto (admin, auth, client, store), conteniendo su respectiva pareja de archivos .html y .ts.
- **src/utls/**: Servicios del sistema (autenticación, persistencia, llamadas simuladas y enrutado).
- **src/types/**: Definición de contratos e interfaces de TypeScript (usuarios, productos, pedidos, roles).

Patrones de Diseño Utilizados

- **Repository / DAO Genérico (BaseRepository<T>)**: Abstrae las operaciones básicas CRUD comunes a todas las entidades (guardar/merge, buscar por ID, listar activos y eliminar de forma lógica), evitando la duplicidad de código en repositorios individuales.
- **DTO (Data Transfer Object)**: Utilizado en clases como UsuarioDTO y PedidoDTO para transferir datos estructurados sin exponer la representación interna ni el estado administrado de las entidades JPA.
- **Builder (Lombok @SuperBuilder)**: Utilizado para simplificar y dar legibilidad a la instanciación de entidades con múltiples campos opcionales u obligatorios.

Manejo de Excepciones y Validaciones

- **DatosException**: Excepción personalizada de negocio para reportar errores de validación de datos (campos requeridos vacíos, stocks insuficientes, precios inválidos).
- **JPAException**: Encapsula errores de persistencia a nivel de base de datos, abstrayendo las excepciones nativas de Jakarta/Hibernate del flujo de control de la consola.
- **OperacionCanceladaException**: Controla la cancelación voluntaria de flujos de carga por parte del operador (mediante el ingreso del caracter *).

Estructura de Base de Datos y Persistencia

La base de datos se autogenera y sincroniza a partir de las anotaciones de las entidades de Java mediante la propiedad *hibernate.hbm2ddl.auto = update*.

Las relaciones principales son:

- Categoría 1 → * Producto (Agregación unidireccional).
- Usuario 1 → * Pedido (Asociación unidireccional).
- Pedido 1 → * DetallePedido (Asociación unidireccional).
- Pedido * → 1 Producto (Asociación unidireccional).

Referencia de Interfaces y Pantallas

- **Pantalla de Login:** Formulario de validación de accesos con autenticación local.

FOOD STORE

Iniciar Sesión

Email
ejemplo@ejemplo.com

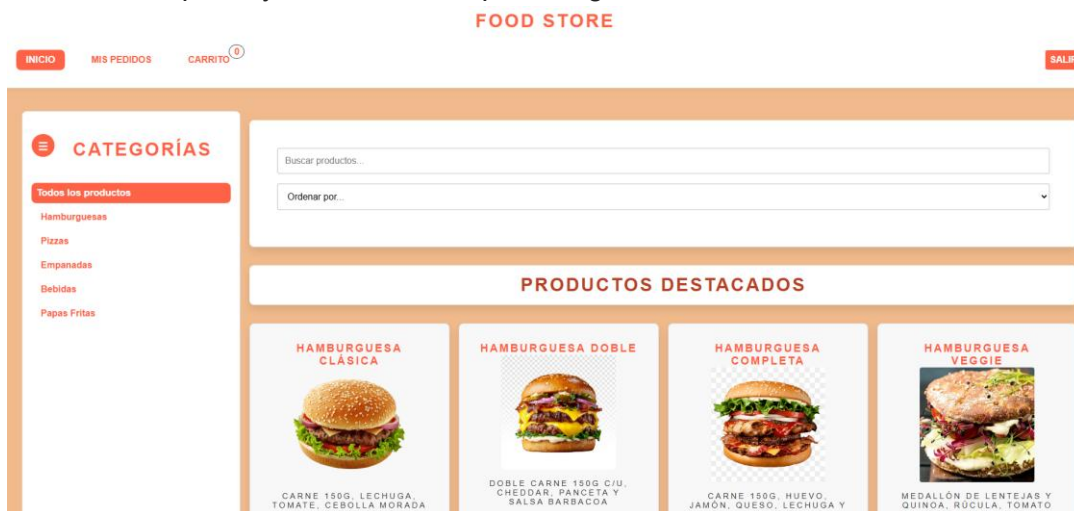
password

INGRESAR

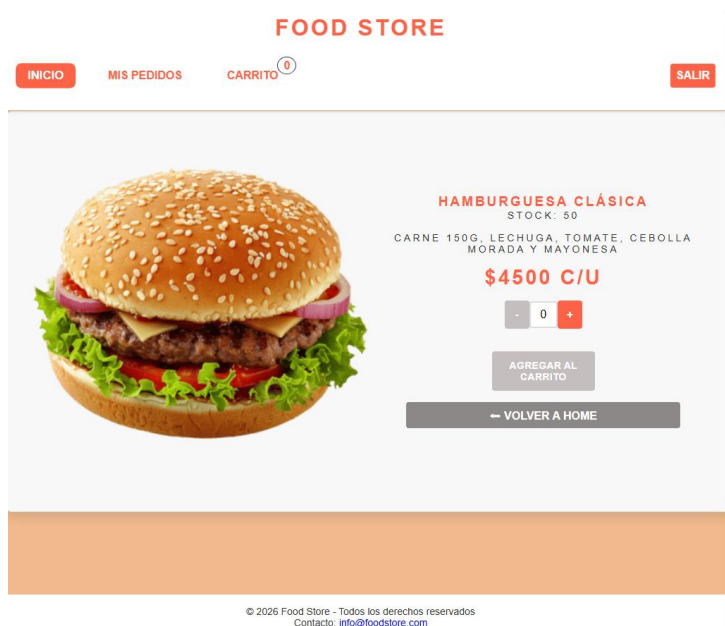
¿No tenés cuenta? [Regístrate](#)

Cuentas de Prueba:
Admin: admin@admin.com / 123456
Cliente: cliente@food.com / cliente123

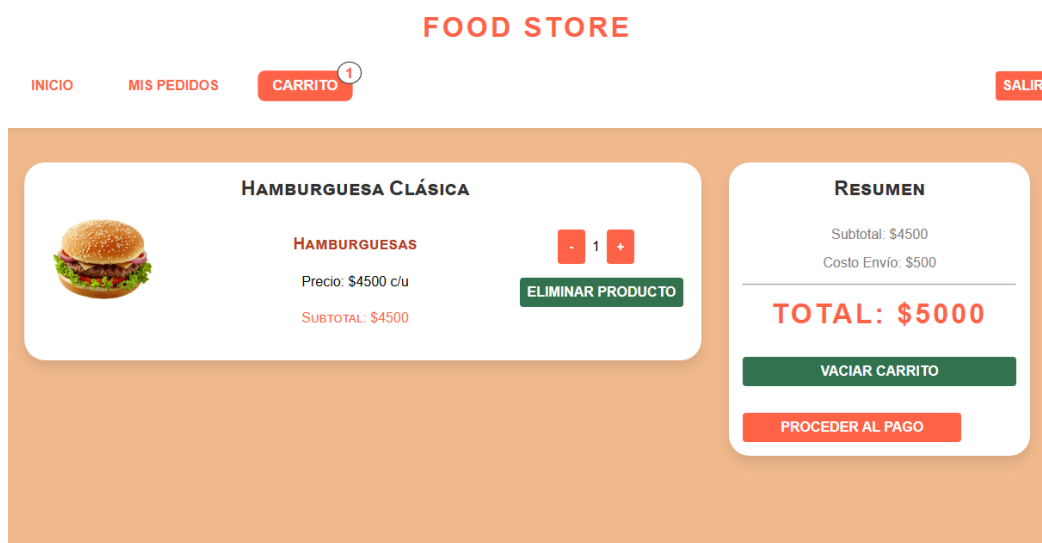
- **Vista de Catálogo (Home Store):** Muestra los productos organizados en cuadrículas con barra de búsqueda y filtros laterales por categoría.



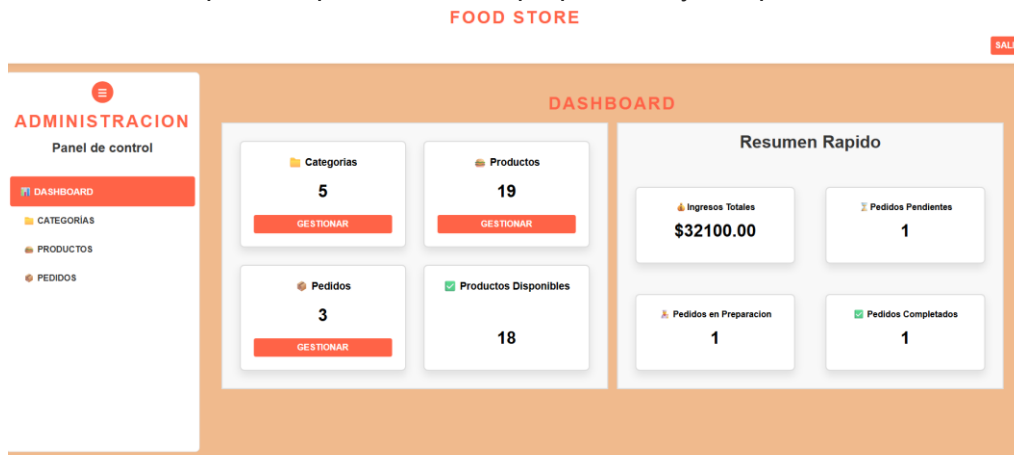
- **Detalle de Producto:** Permite elegir cantidades validadas según stock remanente y agregar al carrito.



- **Carrito de Compras:** Lista interactiva que permite modificar unidades o eliminar productos antes de confirmar la orden de compra.



- **Dashboard del Administrador:** Panel estadístico que muestra el resumen rápido de ventas totales, pedidos pendientes, en preparación y completados.



- **Panel principal consola administración Backend:** Permite acceder a las operaciones principales de gestión de Categorías, Productos, Usuarios y Pedidos; así como la visualización de reportería sobre los datos existentes.

```
MENU PRINCIPAL =====  
  
1. Menu Categorias.  
2. Menu Productos.  
3. Menu Usuarios.  
4. Menu Pedidos.  
5. Menu Reportes.  
0. Salir.  
  
Ingresa una opcion:
```

Dificultades y Soluciones

Concurrencia y Ciclo de Vida del EntityManager

Inicialmente, el proyecto implementaba una única variable estática compartida EntityManager dentro de una clase utilitaria singleton (JPAUtil). Aunque funcionaba para un único usuario secuencial, esto es una mala práctica, ya que EntityManager no es thread-safe y en un sistema real, podría generar colisiones de transacciones. Tuve que refactorizar la clase JPAUtil para remover el campo estático y exponer únicamente la fábrica EntityManagerFactory. A partir de esto, cada método en los repositorios crea su propia instancia local de EntityManager y asegura su cierre explícito en un bloque finally:

```
EntityManager em = JPAUtil.getEntityManagerFactory().createEntityManager();
try {
    // Operaciones de base de datos
} finally {
    if (em != null && em.isOpen()) em.close();
}
```

Este fue un error que tuve durante el trabajo práctico anterior y aproveché esta instancia para corregirlo.

Atomicidad de Transacciones Complejas

El alta de pedidos requiere múltiples pasos críticos: registrar el pedido, crear múltiples detalles de pedido con subtotales correctos, y restar del stock actual del producto. Si uno de los productos de la lista se quedaba sin stock a mitad del proceso, o fallaba la base de datos, el sistema podía quedar en un estado inconsistente. Se delegó este flujo complejo a un método específico transaccional dentro del repositorio: PedidoRepository.registrarPedido(). El método abre un único EntityManager, inicia una sola transacción atómica, recupera las entidades del mismo contexto de persistencia, realiza las validaciones de negocio e impacta todos los cambios de forma conjunta. Si cualquier paso falla, se captura la excepción y se ejecuta un rollback() completo:

```
try {
    em.getTransaction().begin();
    // Carga de entidades y reducción de stock
    em.persist(pedido);
    em.getTransaction().commit();
} catch (Exception e) {
    if (em.getTransaction().isActive()) em.getTransaction().rollback();
    throw e;
}
```

Esta solución, aunque funcional, rompe ciertas reglas de calidad de código, por el nivel de complejidad del método generado. Con una relación Eager, o con un merge para reconectar las entidades relacionadas, podría haber evitado el code smell, pero intenté respetar las consignas establecidas.

Mapeo de Relaciones Bidireccionales JPA

Al modificar las relaciones bidireccionales por unidireccionales, se tuvieron que ajustar todos los modelos, y también encontrar alternativas para resolver algunas cuestiones de acceso a datos que, con la bidireccionalidad, se habían resuelto de forma directa. Para poder acceder a los datos de una entidad contenedora, se evaluaron distintas opciones, tales como hacer un fetch directamente en los queries, lo cual hubiera roto la propiedad explícita, solicitada por consigna, Lazy de las entidades; también se barajó armar mapas desde un query separado al de consulta

de la entidad, en donde se guardase la relación entre la entidad contenida y la contenedora y al final se terminó optando por una consulta por separado de la entidad padre. Un ejemplo, podría ser el producto, consulta su categoría. Estoy consciente del impacto que conlleva esta decisión, pero opté por respetar el modelo propuesto.

Correcciones por Inconsistencias entre JSON de datos y tipos preexistentes

Al comenzar a desarrollar el frontend, se observaron algunas inconsistencias en las estructuras provistas y los tipos generados para el trabajo práctico anterior. Para poder continuar, se realizaron las modificaciones correspondientes en dichas estructuras, así como en los JSON.

Conclusiones

Con el desarrollo de *Food Store* pude integrar, al menos conceptualmente, tecnologías Frontend y Backend. Me permitió abordar los requerimientos desde ambos frentes, pudiendo visualizar la totalidad del modelo, y teniendo que elegir soluciones que satisficieran las necesidades de ambos. Pude aplicar persistencia con JPA, optimización de consultas con JPQL y manejo de transacciones, adoptando buenas prácticas como programación genérica y borrado lógico. Aún habiendo trabajado anteriormente con estas tecnologías, el haber aplicado el modelo solicitado me permitió profundizar en los conceptos y la diversidad que genera un cambio que, a primera vista, uno podría pensar que es pequeño, pero que puede conllevar a orientar el desarrollo por caminos bien diferenciados. Gracias al diseño del sistema proporcionado, estructurado en capas independientes y aisladas para el acceso a datos, llevar a cabo una futura integración mediante API REST pareciera ser una tarea sencilla.

Bibliografía / Webgrafía

1. Documentación Oficial de Java SE: <https://docs.oracle.com/en/java/>
2. Jakarta Persistence API Specification: <https://jakarta.ee/specifications/persistence/>
3. Hibernate ORM Documentation: <https://hibernate.org/orm/documentation/>
4. H2 Database Engine Manual: <https://www.h2database.com/html/manual.html>
5. Vite - Guía de Inicio Rápido: <https://vite.dev/guide/>
6. TypeScript - Documentación Oficial: <https://www.typescriptlang.org/docs/>

Link a YouTube

<https://youtu.be/iQQCM7DvxXs>

Link a repositorios

Backend: https://github.com/mauro-cuquejo-tupad/Cuquejo_Mauro_TPI

Frontend: <https://github.com/mauro-cuquejo-tupad/cuquejo-tpi-programacion-3-front>